

Designing AI-Enabled Solutions

LLMs • Prompt Engineering •
Prompt Management

A practical decision framework for business and technical teams

- 1 Choose the right problem
- 2 Understand the model
- 3 Engineer the prompt
- 4 Manage quality at scale

Enterprise AI • Banking-oriented examples

Three questions guide the chapter

From business need to governed AI solution

1. Should we use AI?

Frame the problem, inspect the data, understand the users and workflow, evaluate risks, and define measurable success.

2. What can the LLM really do?

Understand tokens, context, training, inference, strengths and failure modes. Design around limits rather than hoping they disappear.

3. How do we make it reliable?

Use structured prompts, examples, schemas, test sets, versioning, evaluation and controlled deployment.

The same decision framework will be reused across later AI use cases in the course.

AI is a means, not the objective

Start with the business outcome, not with the model

A weak starting point

“We need a chatbot.”

A stronger starting point

“Reduce average handling time for mortgage-document questions by 30% while preserving compliance.”

Technology comes later

AI, rules, workflow redesign, search, analytics—or a combination.

The baseline matters

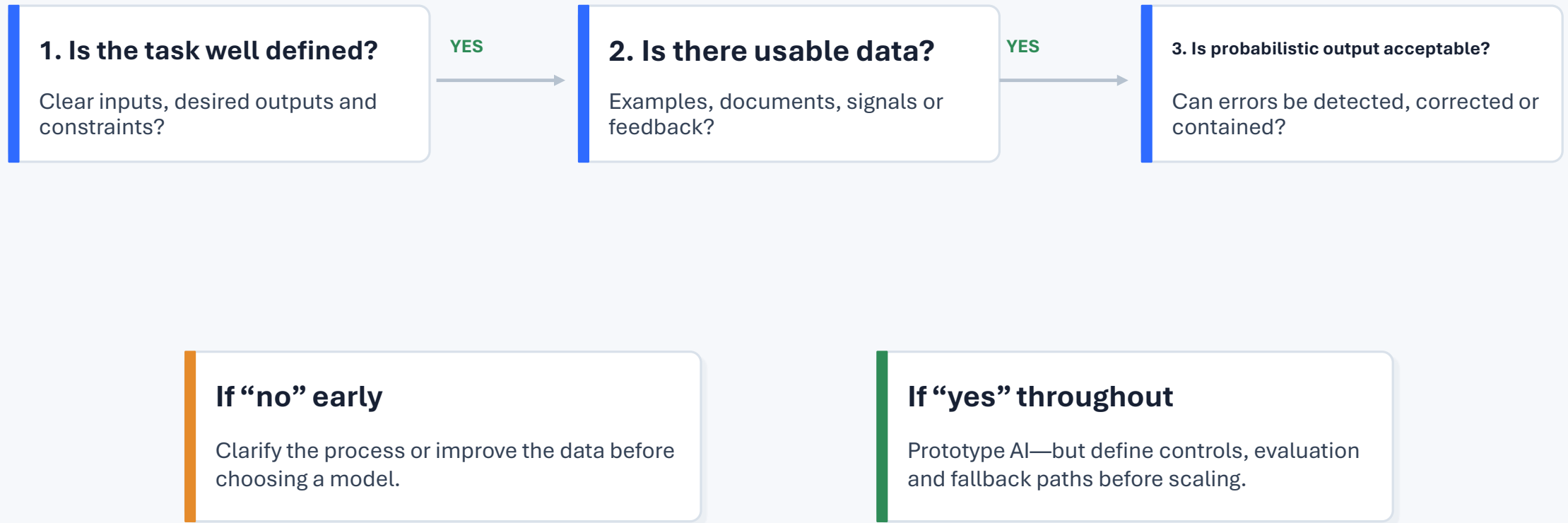
Compare against the current process, not against a perfect hypothetical system.

Value must be observable

A model is useful only if it improves a user or business outcome.

When is AI the right answer?

A first-pass decision tree

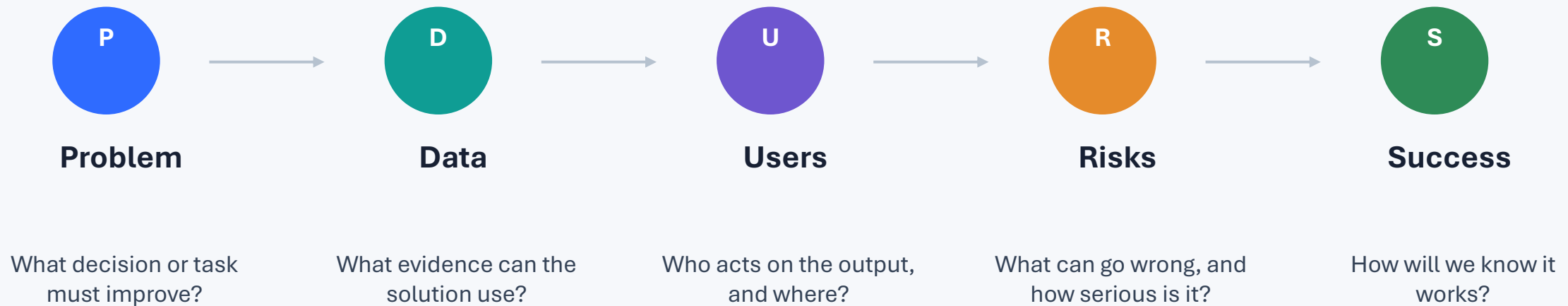


Strong AI candidates often involve unstructured information, fuzzy patterns, language, images, prioritization or prediction.

PROBLEM!

DECISION FRAMEWORK The common use-case framing canvas

Five lenses used throughout the course



Do not move to “Which model?” until these five lenses are explicit.

1. Frame the problem as a measurable change

Avoid solution language in the first sentence

Current state

What happens today? Who does what, with which tools and delays?

Target state

What should become faster, safer, cheaper or more consistent?

Decision boundary

Which part needs judgment, language understanding or pattern recognition?

Non-goals

What is explicitly out of scope for the first release?



2. Assess data fitness before model selection

Availability is not the same as usability

Coverage

Does the data represent the cases we care about?

Quality

Is it accurate, complete and current enough?

Access

Can the solution legally and technically retrieve it?

Labels / feedback

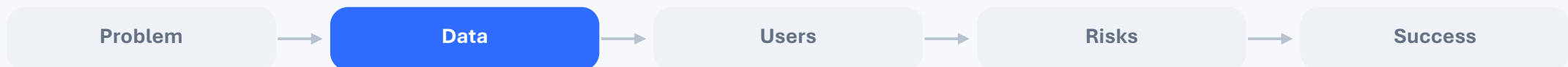
Can we measure correctness or gather user feedback?

Sensitivity

Does it contain personal, confidential or regulated data?

Drift

Will the data or process change over time?

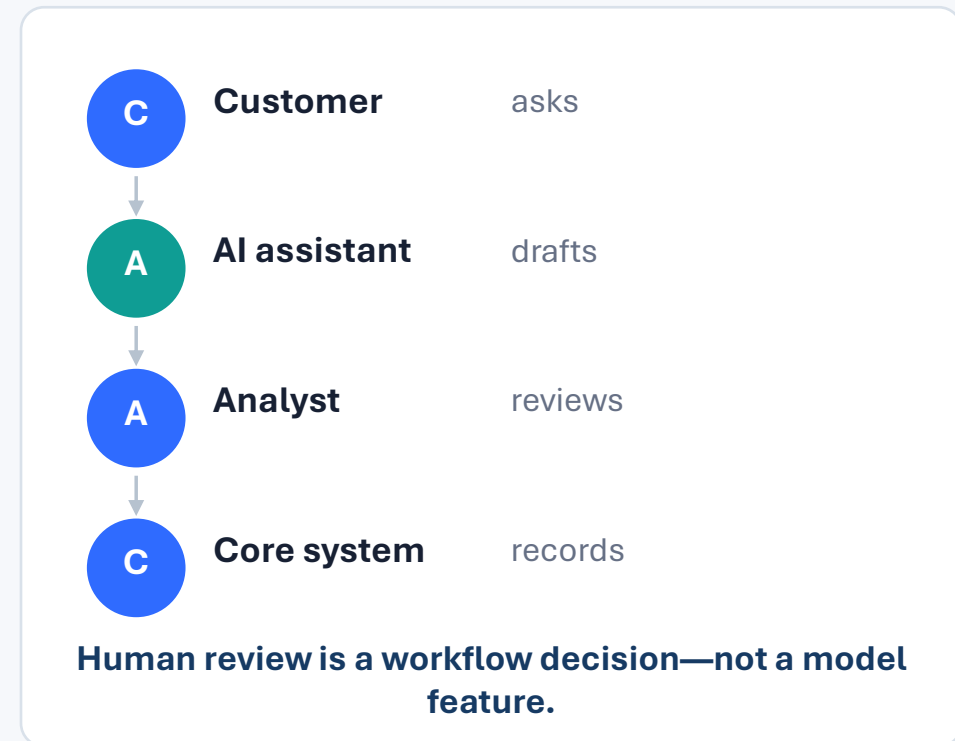


3. Design for the user and the workflow

A good model in the wrong workflow still fails

Ask four workflow questions

- Who initiates the task?
- Who reviews or approves the output?
- What happens when confidence is low or information is missing?
- Where is the output consumed: email, CRM, case-management, API, desktop app?



4. Map risks before prototyping

Probability × impact × detectability

Wrong answer

Model error or hallucination

Data leakage

Sensitive content exposed or logged

Bias / unfairness

Different outcomes across groups

Automation bias

Users over-trust fluent output

Security abuse

Prompt injection, data exfiltration

Operational failure

Latency, cost, outage or tool dependency

Problem



Data



Users



Risks



Success

Risk severity determines the required controls, review level, logging and fallback path.

5. Define success metrics at three levels

Model quality is necessary—but not sufficient

Model / output metrics

- Accuracy / groundedness
- Completeness
- Format validity
- Safety violations
- Calibration / confidence

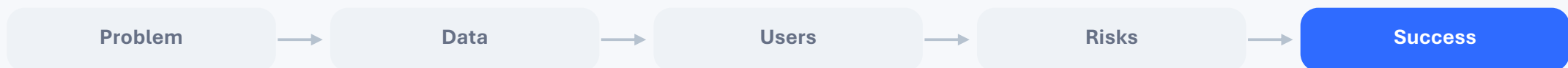
Workflow metrics

- Handling time
- First-pass resolution
- Escalation rate
- Rework rate
- User acceptance / override

Business metrics

- Cost per case
- Revenue or conversion
- Risk reduction
- Customer satisfaction
- Compliance outcomes

A successful AI system improves the process without creating unacceptable new risk.



DECISION FRAMEWORK

A reusable “AI or not?” scorecard

Use it consistently before comparing vendors or models

Business value Is the outcome valuable enough to justify change?

Low

Med

High

Data readiness Is there enough usable evidence?

Low

Med

High

AI advantage Does AI outperform simpler alternatives?

Low

Med

High

Risk controllability Can errors and abuse be contained?

Low

Med

High

Integration fit Can it operate in the real workflow?

Low

Med

High

Measurability Can success be tested objectively?

Low

Med

High

The scorecard is not a mathematical truth: it is a structured conversation that exposes assumptions early.

Example: AI-assisted KYC document review

Using the five-lens framing canvas

- 1 Problem** Reduce manual reading of onboarding documents while preserving analyst control.
- 2 Data** Forms, IDs, company documents, policy rules, historical review outcomes.
- 3 Users** KYC analyst reviews extracted facts and model-generated summaries.
- 4 Risks** Missed red flags, hallucinated facts, privacy leakage, false confidence.
- 5 Success** Lower handling time with no increase in compliance exceptions.

Likely solution: document extraction + retrieval + LLM summarization + human review—not “LLM only”.

Sometimes the best AI decision is “do not use AI”

Prefer simpler methods when they are more reliable, transparent or cheaper

Exact calculations

Interest, fees, accounting totals → deterministic formulas

Stable eligibility rules

Clear policy logic → rule engine / decision table

Small structured lookup

Known facts → database query / search

Rare high-impact decisions

If errors are unacceptable and cannot be reviewed → keep deterministic / human control

Use AI when it adds unique value

- Understanding messy language or documents
- Finding patterns that rules cannot enumerate
- Generating or transforming content
- Prioritizing cases under uncertainty

Rule of thumb: deterministic where possible, probabilistic where valuable.

What is an LLM?

A model trained to predict the next token from context



Language model

Learns statistical structure of text and other tokenized modalities.

Large

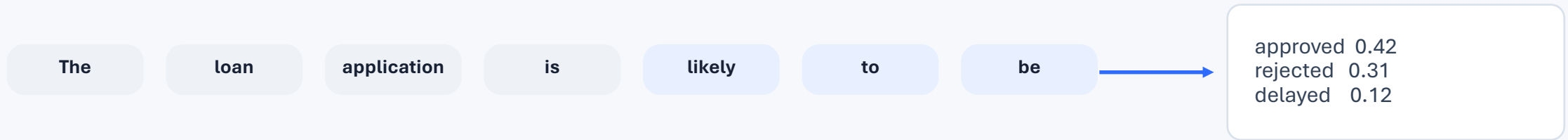
Many parameters and large-scale training data / compute.

Generative

Produces new sequences one token at a time.

Generation is repeated next-token prediction

One step is simple; the sequence becomes powerful



Sample / select one token → append it → repeat



This explains both fluency and uncertainty: the model generates plausible continuations, not guaranteed facts.

Tokens are the model's basic units

Text is converted to token IDs before the model sees it

Example text

“International banking regulations”

Possible tokenization

International

bank

ing

regulations

Why tokens matter

Context limits, cost and latency are usually measured in tokens—not words.

Tokenization is not linguistic truth

Words may split into several tokens; punctuation and spaces can also matter.

Multimodal models tokenize more than text

Images, audio and other inputs are represented as model-readable units too.

The context window is the model's working memory

Everything the model can consider in one inference call

System / policy

Conversation

Retrieved docs

User input

Output budget

More context can help

Longer documents, richer history and more retrieved evidence can improve relevance.

But more is not always better

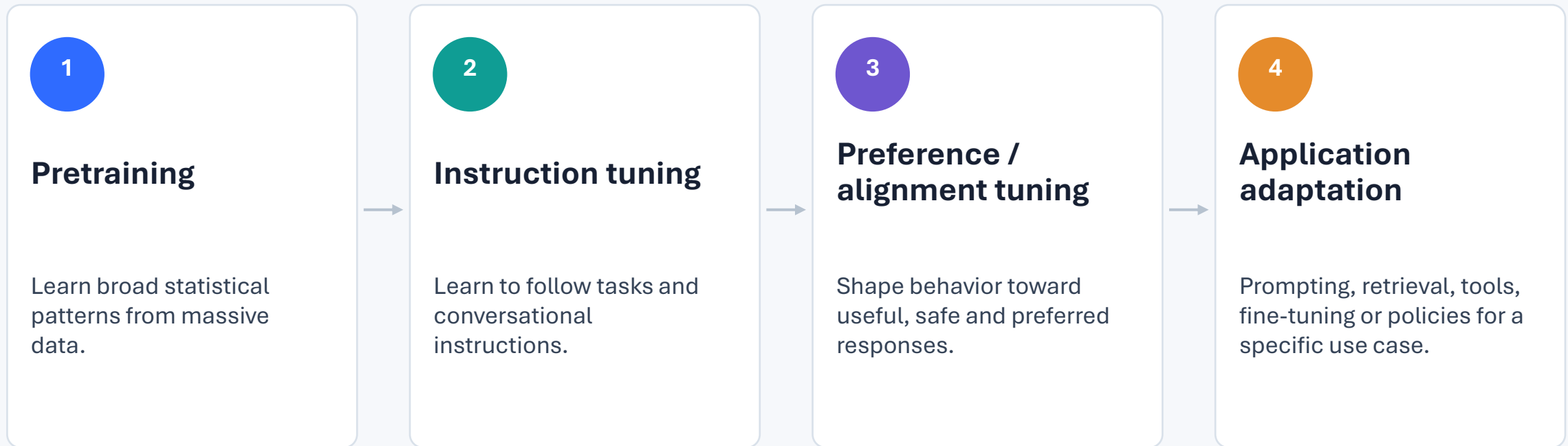
Irrelevant context can distract the model and increase cost and latency.

Context is not persistent memory

Unless the application stores and re-injects information, a new call does not “remember” it.

From foundation model to useful assistant

Conceptual training stages



Most enterprise teams consume pretrained models and focus on application adaptation—not training from scratch.

Inference: what happens when the application calls the model

Prompt in → tokens processed → output generated



Inference design choices

Model

Temperature

Max output

Tools

Retrieval

Safety filters

Timeout / retry

Logging

What LLMs are good at

Capabilities are strongest where language patterns and transformation matter

Summarize

Condense documents, conversations or cases.

Extract

Turn unstructured text into fields and entities.

Classify

Route, tag or prioritize textual cases.

Generate

Draft emails, reports, explanations and content.

Transform

Rewrite, translate, normalize tone or format.

Reason with support

Compare evidence, follow procedures, use tools and retrieved context.

Capability does not imply reliability. Each use case still needs task-specific evaluation and controls.

Why LLMs changed enterprise AI

They make unstructured information programmable

Before

- Each NLP task required a specialized model or rules.
- Interfaces were narrow and expensive to extend.
- Unstructured data often stayed outside core automation.

With modern LLMs

- One model can handle many language tasks.
- Natural-language instructions lower the cost of experimentation.
- Tools, retrieval and structured outputs connect language to enterprise systems.
- The same platform can support many workflows.

Hallucinations: fluent output can still be wrong

The model optimizes plausibility, not truth

Typical causes

- Missing or ambiguous information
- Weak or conflicting context
- Prompt asks beyond available evidence
- Sampling variability
- Model knowledge is outdated or incomplete

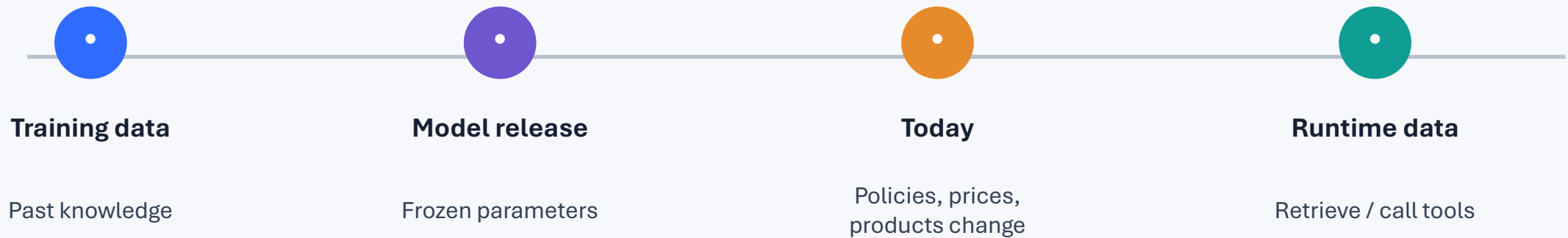
Design responses

- Ground answers in trusted sources
- Require citations or source IDs where useful
- Allow “insufficient information” as a valid output
- Use deterministic validation for critical fields
- Escalate high-risk cases to humans

Do not “prompt away” hallucinations. Reduce their probability and limit their impact.

Knowledge cut-off and changing facts

Static model knowledge is not a reliable source for current enterprise facts



Retrieval

Inject current internal documents or indexed knowledge.

Tools / APIs

Query live systems for balances, rates, status or rules.

A layered control model for LLM applications

No single guardrail is enough

5 Monitoring & audit

Logs, quality trends, incidents, drift

4 Human / workflow controls

Review, approval, escalation, fallback

3 Output controls

Schema validation, citations, business rules

2 Model controls

Model choice, temperature, safety configuration

1 Input controls

Access, redaction, prompt-injection defenses

Prompt engineering is interface design for a probabilistic model

A prompt specifies task, context and output contract

Weak prompt

“Summarize this document.”

Ambiguous audience • unclear length • no required fields • no handling for missing data

Engineered prompt

- Role: “You are a credit analyst assistant.”
- Task: “Summarize decision-relevant facts only.”
- Constraints: “Use only the supplied document.”
- Output: “Return JSON with 6 required fields.”
- Fallback: “Use null when evidence is missing.”

A practical prompt structure

Role + instructions + context + examples + output format

ROLE

Who the model should act as and for whom.

INSTRUCTIONS

What to do, what not to do, priorities and constraints.

CONTEXT

Data, documents, definitions, policies or retrieved evidence.

EXAMPLES

Demonstrations of desired behavior and edge cases.

OUTPUT FORMAT

Schema, sections, fields, types and acceptance rules.

Not every prompt needs every block, but the structure is a useful debugging checklist.

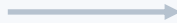
Examples teach behavior more precisely than prose alone

Few-shot prompting is especially useful for edge cases

Instruction only

“Classify each complaint as billing, fraud, card, mortgage or other.”

Problem: ambiguous boundaries remain hidden.



Instruction + examples

“ATM kept my card after PIN retries.”

card

“I do not recognize this transfer.”

fraud

“Early repayment fee is wrong.”

mortgage

Examples define category boundaries and desired wording.

Structured outputs turn text generation into system integration

BANKING EXAMPLE

Ask for machine-readable data with an explicit schema

```
{  
  "customer_id": "C01842",  
  "risk_level": "medium",  
  "missing_documents": [  
    "proof_of_address"  
  ],  
  "requires_review": true,  
  "evidence": ["page_4", "page_7"]  
}
```

Why it helps

Easier validation, routing, storage and downstream automation.

What to specify

Field names, data types, allowed values, null behavior and required evidence.

What to validate

Schema compliance plus business rules. A valid JSON object can still be factually wrong.

Treat prompts as versioned, testable assets

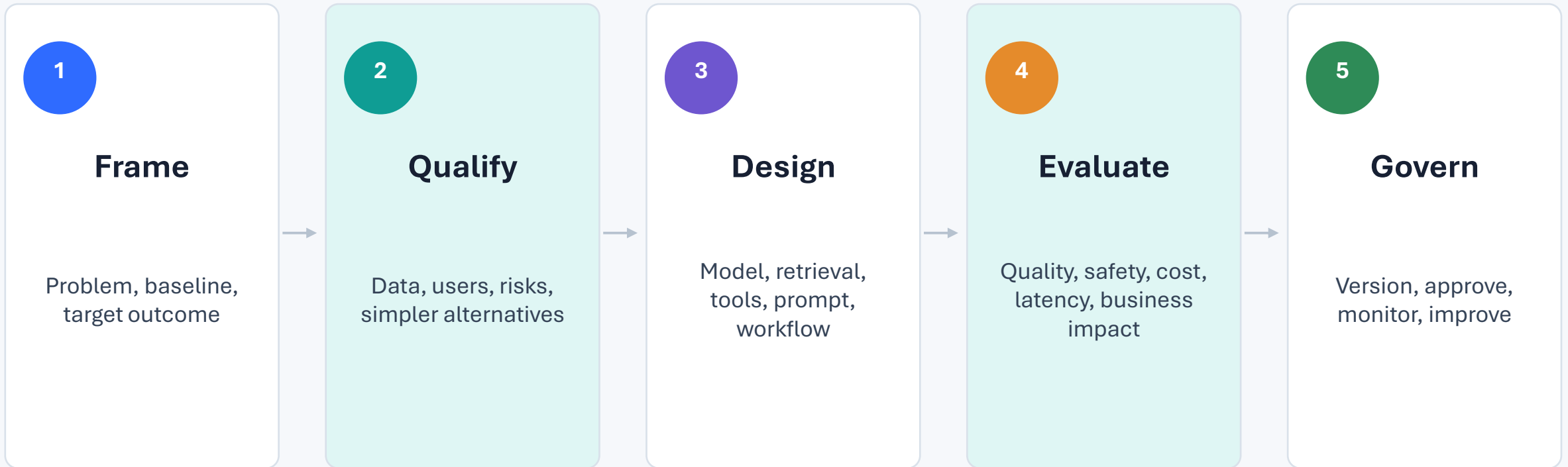
Prompt management is software engineering for model behavior



Never change a production prompt without knowing what changed, why it changed, and how it performed on the same evaluation set.

The course decision framework: from idea to governed AI solution

Use this checklist for every new use case



Key principle Choose AI only when it creates measurable value and its uncertainty can be controlled.